# PROIECTARE INTERFEȚE UTILIZATOR

## Lucrare de laborator nr. 2

### Cuprins


1. Obiective ..... 2
2. Însușirea modului de lucru cu clase în Visual C# ..... 2
3. Proprietăți ..... 2
4. Tipuri referință de date ..... 3
5. Temă de laborator ..... 4
6. Temă acasă ..... 5
7. Bibliografie ..... 5

# 1. Obiective

- Însușirea modului de lucru cu clase în Visual C#
- Proprietăți
- Tipuri referință de date (tablouri, șiruri de caractere)

# 2. Însușirea modului de lucru cu clase în Visual C#

Parcurgeți în modul Debug proiectul ***ExempluClase*** si urmăriți:

- Modul de scriere al clasei in fișier separat;
- Modul de definire al constructorilor;
- Modul de inițializare a obiectelor de tip **FiguraGeometrica.**

# 3. Proprietăți

Proprietățile pot fi de 2 tipuri:

- proprietăți **auto-implemented;**
- proprietăți **computed** (calculate pe baza altor date membre).

Analizați aplicația ***ExempluClase***:

- Observați proprietățile auto-implemented și modul lor de utilizare.
- Analizați în cadrul articolului [1] diferența dintre date membre și proprietăți.

```
// proprietati auto-implemented
public string Denumire { get; set; }
public int NrLaturi { get; set; }

// proprietate computed – varianta 1
public bool EstePoligon
{
    get
    {
        return NrLaturi >= 3;
    }
}

// proprietate computed – varianta 2 (expresie Lambda)
public bool EstePoligon_v2 => NrLaturi >= 3;
```

# 4. Tipuri referință de date

## 4.1. Vectori

**Observații**:

- În C# vectorii sunt obiecte.
- Simpla declarare a unui vector nu alocă și spațiu de memorie.
- Proprietatea Length furnizează numărul de elemente din vector.
- Transmiterea vectorilor ca argumente ale unor funcții se face prin referință (dacă metoda modifică valorile elementelor vectorului, aceste modificări se regăsesc și în codul apelant).
- Observați modul de expunere a datelor membre de tip vector in cadrul clasei **FiguraGeometrica** din aplicatia **ExempluClase**.

```
// Declararea unui vector de intregi
int[] vector;
// Alocarea memoriei pentru vectorul declarat
vector = new int[3];

/* Declararea și initializarea unui vector de intregi, fara a fi necesara specificarea
   explicita a dimensiunii */
int[] vectorIntregi = { 1, 3, 2, 7, 6 };

// Proprietatea Length – furnizeaza numarul total de elemente din vector
int lungimeVector = vector.Length;

/* Declararea unui vector de numere reale
   si alocarea memoriei pentru 3 elemente neinitializate explicit */
float[] vectorNumereReale = new float[3];
vectorNumereReale [0] = 1;
vectorNumereReale [1] = 2.5;    /* Eroare de compilare!!!. C# nu poate converti
                  în mod automat un double la float
                  Modalitati de initializare corecta a datelor de tip float */
vectorNumereReale [1] = 2.5F;
vectorNumereReale [2] = (float)3.2;
```

```
// Declararea unui vector de siruri de caractere
string[] vectSir;
// Alocarea memoriei pentru vectorul cu maxim 100 de elemente siruri de caractere
vectSir = new string[100];

// Declararea si initializarea unui vector de siruri de caractere
string[] numePersoane = { "Ionescu", "Ioan", "Vasilescu" };

// Utilizarea instructiunii foreach pentru accesarea elementelor unui vector
```

```
foreach(string nume in numePersoane)
{
    Console.WriteLine(nume);
}
```

Rezultat execuție în consola:

```
Ionescu
Ioan
Vasilescu
```

## 4.2. Șiruri de caractere

```
//  Exemplu utilizare metoda 'split'
string propozitie = "Program cu siruri de caractere.";
string[] cuvinte = propozitie.Split(' ');

foreach (string cuvant in cuvinte)
{
    Console.WriteLine("{0}", cuvant);
}

//  Exemplu utilizare metoda 'join'
string propozitieModificata = string.Join("#", cuvinte);
Console.WriteLine("Propozitie modificata: {0}", propozitieModificata);
```

Rezultat execuție în consola:

```
Program
cu
siruri
de
caractere.
Propozitie modificata: Program#cu#siruri#de#caractere.
```

# 5. Temă de laborator

1. Modificați metoda ***Info*** a clasei ***FiguraGeometrica*** din aplicația ***ExempluClase*** astfel încât, atunci când denumirea nu conține nici un caracter, să se afișeze mesajul „FIGURA NESETATA”.
2. Implementați o secvența de cod de preluare a numărului de laturi si a dimensiunilor laturilor figurii de la tastatura si creați un obiect de tip FiguraGeometrica având caracteristicile respective.
3. Afișați dimensiunile laturilor cu spatii între ele in cadrul metodei **Info**.

4. Adăugați clasei ***FiguraGeometrica*** o proprietate ***Perimetru*** care să calculeze perimetrul figurii prin adunarea dimensiunilor laturilor.
5. Adăugați o proprietate ***TipFigura*** în clasa ***FiguraGeometrica*** care să returneze tipul figurii în funcție de numărul de laturi introduse (Indicație: utilizați structura de control ***switch***). Modificați metoda ***Info*** pentru a afișa atât denumirea introdusă de utilizator cât și denumirea determinată în funcție de numărul de laturi.
6. Realizați o aplicație tip joc în care introduceți numărul de laturi și afișați denumirea figurii geometrice. Generați într-o structura repetitivă cate o valoare pentru numărul de laturi, formați obiecte de tip ***FiguraGeometrica*** cu denumire = sir vid si număr de laturi = numărul generat si afișați tipul figurii formate.

## 6. Temă acasă

1. Identificați clasa (clasele) necesare modelării entităților din cadrul temei alese și implementați-le utilizând limbajul C#.
2. Folosiți un software de versionare a fișierelor pentru a publica clasele proiectate si aplicația de test.

## 7. Bibliografie

[1] Diferența dintre date membre și proprietăți - https://medium.com/@kamaleshs48/difference-between-field-and-property-34c122aa429c

[2] GitHub Extension for Visual Studio: https://visualstudio.github.com/

[3] Tutorial GitHub Extension for Visual Studio: https://www.c-sharpcorner.com/article/install-and-use-github-extension/